

Student Designed Lab Project: CO₂ and CFC Sensors Lab Notebook

Karina Winn, 7/12/2023

Designing the Experiment

My group, including Melissa Griffin, Adam Packard, Alli Potter, Quinn Roberts, and myself, decided to measure carbons dioxide and chlorofluorocarbons (CFCs) in the atmosphere. This was mainly of interest to us because there isn't a lot of atmospheric data out there for CO₂ or CFCs, and having that data would be useful. Carbon dioxide in the atmosphere is one of the main contributing factors to climate change, and having CO₂ concentrations as a function of altitude would allow us to use wind patterns to determine where that CO₂ is coming from. Chlorofluorocarbons in the atmosphere break down ozone that is protecting us from the sun's UV rays, and they were banned around the world in the 1970s, It would be beneficial to know if those policies reduced the CFC concentration in the atmosphere, and hopefully by now that concentration is close to zero. Our plan for collecting the data is to obtain the sensors, wire and write code for them, and attach them to the weather balloon that the High Altitude Research Team (HART) is launching. Once we have data, we should be able to interpolate it with the altimeter data HART is collecting and compare it to accurate data to confirm our findings. We do however have some concerns. The CFC sensor that we are planning to purchase, the MQ-138, detects 1-100 parts per million, and from a little bit of research on the internet, we expect the CFC concentration in the atmosphere to be less than that. This means that it might not measure anything at all. We do expect the CO₂ sensor, a SEN0220 from DFRobot, to work just fine.

Performing the Experiment

As a group we have a lot of things to get done, so we divided the tasks amongst ourselves to get them done as quickly as possible. This means that there are some parts of the project I didn't work on, so I will direct you to the lab notebooks of those who did work on them.

CO₂ Sensor

The CO₂ sensor is one of those things I did not work on. You can look at Melissa Griffin and Alli Potter's notebooks to see how they got the CO₂ sensor up and running.

CFC Sensor

Quinn Roberts and I worked on the CFC sensor together.

The first step before doing anything else is to do some research. Our main goals were to find code for our sensor and data that we could compare our data to later. We couldn't find data to compare to, but in the process of researching I did find that the sensor needs to be calibrated for 2 hours before taking any measurements, which is something we can do beofre the balloon launch.

After doing our research, we were ready to put the sensor together. We were able to find a wiring diagram with some example code, which can be found through the following link:

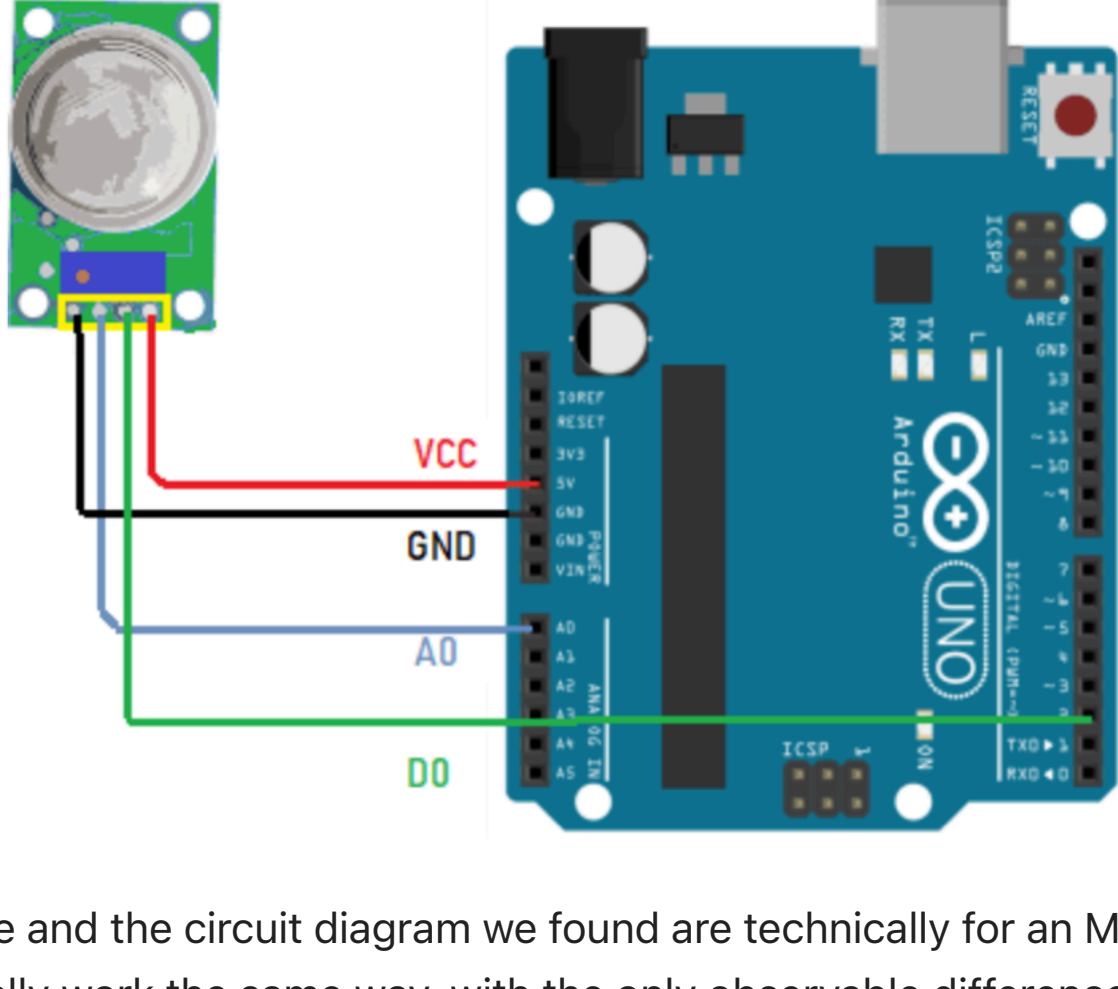
<https://microcontrollerslab.com/interfacing-mq-135-gas-sensor-arduino/>

This is the code we got from that site:

```
In [ ]: 1 int sensorValue;
2 int digitalValue;
3
4 void setup()
5 {
6   Serial.begin(9600); // sets the serial port to 9600
7   pinMode(13, OUTPUT);
8   pinMode(2, INPUT);
9 }
10
11 void loop()
12 {
13   sensorValue = analogRead(0); // read analog input pin 0
14   digitalValue = digitalRead(2);
15   if (sensorValue > 400)
16   {
17     digitalWrite(13, HIGH);
18   }
19   else
20     digitalWrite(13, LOW);
21   Serial.println(sensorValue, DEC); // prints the value read
22   Serial.println(digitalValue, DEC);
23   delay(1000); // wait 100ms for next reading
24 }
```

After taking a look at this code, we determined that the digital output was only being used to turn an LED on and off, which isn't something we're going to be able to see when our sensor is on the balloon. Knowing this, we were able to delete lines 2, 14-20, and 22.

Next we wired our instrument. Our code is eventually going to save data to an SD card with a time stamp, so we added a data shield to our arduino. Then we wired our sensor according to the circuit diagram found at the above link.



An important thing to note. Both the code and the circuit diagram we found are technically for an MQ-135 sensor. However, we found that the MQ-135 and MQ-138 sensors generally work the same way, with the only observable difference being the position of the pins on the sensor. We adjusted and wired our instrument with pins on the arduino connecting to the right type of pin on the sensor rather than the pin in that position on the sensor.

Our next step is integrating the above code with code that will save our data to an SD card. We were able to find code that Brother Lines wrote that saved data to an SD card, but did it for a different sensor. We copied the code and replaced the code intended for the other sensor with the code above.

This is the final draft of our code:

```
In [ ]: 1 int sensorValue;
2 // === include statements
3 #include <SD.h>
4
5 // RTC library by Adafruit. Must be manually installed
6 #include "RTClib.h"
7
8 // SD card pin to use
9 const int SD_PIN = 10;
10
11 // Real time clock type and variable
12 RTC_DS1307 rtc;
13
14 // pin to blink for light to indicate operation
15 int ledPin = 8;
16 int n = 0;
17
18 // =====
19 // initializes the RTC,
20 // and checks to see if it has been set
21 // =====
22 void init_RTC() {
23   Serial.print("Initializing RTC...");
24   //
25   if (!rtc.begin()) {
26     Serial.println(" failed!");
27     while (1);
28   }
29   //
30   Serial.println(" done!");
31   //
32   if (!rtc.isrunning())
33     Serial.println(
34       "WARNING: RTC has not been previously set");
35 }
36
37 // =====
38 // attempts to initialize the SD card for reading/writing
39 // =====
40
41 void init_SD() {
42   Serial.print("Initializing SD card...");
43   //
44   if (!SD.begin(SD_PIN)) {
45     Serial.println(" failed!");
46     while (1);
47   }
48   //
49   Serial.println(" done!");
50 }
51
52 // =====
53 // SETUP
54 // =====
55
56 void setup() {
57   // Open serial communications for the serial monitor
58   Serial.begin(9600);
59
60   // Start the real time clock. This is from the
61   // Adafruit RTC example code since our data
62   // loggers use Adafruit RTS's
63
64   init_RTC();
65   // (note 24-hour time: 3pm -> 15)
66   // This line sets the RTC with an
67   // explicit date & time, for example:
68   // to set January 21, 2014 at 3:18pm
69   // you would use the following line:
70   // rtc.adjust(DateTime(2014, 1, 21, 15, 18, 0));
71
72   // Initialize the SD card so we can write to it in
73   // the loop
74   init_SD();
75
76   // Now set up your sensor here.
77   Serial.begin(9600); // sets the serial port to 9600
78   pinMode(13, OUTPUT);
79   pinMode(2, INPUT);
80
81   // Tell the User that we started a new collection
82   // Be careful here. Printing things in the setup can be
83   // tricky. Don't print something that is very long
84   File dataFile = SD.open("datalog.txt", FILE_WRITE);
85   dataFile.println("START OF NEW COLLECT");
86   Serial.println("START OF NEW COLLECT");
87   dataFile.close();
88   // set up ledPin for blinking to show operation
89   pinMode(ledPin, OUTPUT);
90 } // End of setup
91
92 // =====
93 // LOOP
94 // =====
95
96 void loop() {
97   // make a place to put our time stamp and data
98   String dataString="";
99   //
100   // get the time stamp information from the RTC
101   DateTime now = rtc.now();
102   //
103   // make an output string that has the time and date
104   // We need to be careful while doing this.
105   // Arduino Unos have only a tiny bit of memory
106   // It is really easy to use it all up and when
107   // we do, the microcontroller can become
108   // unstable. We have three devices connected
109   // to our Arduino, the SD card reader, the RTC
110   // and the sensor, in this case an Adafruit
111   // BME280. That is a lot! So we will output
112   // the data with as few extra characters as
113   // possible. This includes spaces! We will
114   // separate using commas, because python
115   // makes it easy to use csv files. I left
116   // in units, but you could put them in the
117   // header and reduce the dataString further.
118   sensorValue = analogRead(0); // read analog input pin 0
119   dataString = dataString + String(now.year());
120   dataString = dataString + '-' ;
121   dataString = dataString + String(now.month());
122   dataString = dataString + '-' ;
123   dataString = dataString + String(now.day());
124   dataString = dataString + ' ' ;
125   dataString = dataString + String(now.hour());
126   dataString = dataString + ":" ;
127   dataString = dataString + String(now.minute());
128   dataString = dataString + ":" ;
129   dataString = dataString + String(now.second());
130   dataString = dataString + "," ;
131   dataString = dataString + String(sensorValue);
132   dataString = dataString + "ppm";
133
134   // print out the time stamp and the sensor data
135   Serial.println(dataString);
136   File dataFile = SD.open("datalog.txt", FILE_WRITE);
137   dataFile.println(dataString);
138   dataFile.close();
139   //
140   delay(1000); // time between data points
141   if (n==0){
142     digitalWrite(ledPin, HIGH);
143     n=1;
144   }
145   else {
146     digitalWrite(ledPin, LOW);
147     n=0;
148   }
149   // End of Loop
150 }
```

Now that we have everything prepared, it's time to do some ground tests. Before the connections were hot glued, we connected the sensor to a battery and took it up to the chemistry floor and outside to see if we could detect anything. When we returned, we found that the data hadn't been saved to our SD card, which we quickly found was corrupted when we read the files on it. After getting a new SD card, we did a couple more ground tests outside. Once we had determined that the sensor was working, we gave it to Adam Packard to hot glue. We then did another ground test, which was successful. After that, we felt that our sensor was ready to launch.

Flight Preparation and Flight Day

Adam Packard worked on planning and building the box that our sensors would fly in. You can look at his lab notebook for more details on that. I was however at the launch itself. Melissa Griffin and I went to turn our sensors on and make sure that they were working. We then attached them to the balloon and they were launched into the atmosphere at 8:38 am on July 1st. After the balloon landed, I was able to get the data files from off of the two SD cards.

Results and Analysis

After obtaining the data, Alli Potter was able to write code that interpolated both the CO₂ and CFC data with the altimeter data from HART. The resulting graphs were surprising. The CFC sensor collected valid data for the duration of the flight, but the CO₂ sensor did not. It recorded reasonable values for the first and last ten kilometers of the flight, but at heights above ten kilometers it only recorded zeros. The data we did collect from the CFC sensor shows a concentration of about 15 parts per million in the fifteen to thirty kilometer range, which is the ozone layer. With a sensor error of ± 50 parts per million, that value is sure to be either very small or zero.

Conclusions

We were unfortunately unable to draw any conclusions from the CO₂ data, but the CFC data showed that the 1970s ban on CFCs was beneficial to the ozone layer.